

Documentação – Atividade 3: Implementação do Algoritmo A*

Projeto: Projeto Ômega

Integrantes:

- Carlos Eduardo Silva Santos
- Ênio Felipe Teixeira Soares
- Haniel Pereira Albino da Silva
- Renato Souza Gomes

Objetivo da atividade

Nesta atividade, o objetivo foi implementar uma forma de navegação automática para um agente dentro do mapa do Projeto Ômega.

A ideia foi fazer com que o agente conseguisse sair de um ponto inicial e chegar até um destino definido, encontrando o caminho sozinho e evitando as partes do mapa que não podem ser atravessadas.

Para isso, foi utilizado o algoritmo **A***, que é responsável por calcular o caminho mais adequado entre os dois pontos.

Estrutura do mapa

O mapa do jogo foi construído utilizando uma matriz de **15 linhas por 15 colunas**. Cada posição dessa matriz representa um tile do cenário.

Cada tile possui algumas informações que são utilizadas pelo jogo, como o tipo de terreno, se ele pode ser atravessado e o custo para passar por ele.

Os principais tipos utilizados são:

- **FLOOR:** representa o chão normal e pode ser atravessado.
- **WALL:** representa as paredes e não pode ser atravessado.

- **MUD:** representa uma área que pode ser atravessada, mas possui um custo de movimentação diferente.

Essa estrutura também foi utilizada anteriormente para controlar a colisão do jogador com as paredes.

Implementação do A*

Para fazer a navegação do agente, foi criada a classe AStarPathfinder.

O algoritmo A* trabalha analisando os possíveis caminhos entre o ponto inicial e o destino. Para escolher os caminhos, ele utiliza três valores:

$$F = G + H$$

Onde:

- **G** é o custo do caminho percorrido até o momento;
- **H** é uma estimativa da distância até o destino;
- **F** é o resultado da soma dos dois valores.

O algoritmo vai analisando os tiles disponíveis e descartando aqueles que estão bloqueados ou que estão fora dos limites do mapa.

Depois que encontra o destino, o caminho é reconstruído e enviado para o agente seguir.

Heurística

A heurística utilizada foi a **Distância de Manhattan**.

Ela foi escolhida porque o agente não se movimenta na diagonal. Ele pode andar somente para cima, baixo, esquerda e direita.

O cálculo utilizado considera a diferença entre a linha e a coluna da posição atual e do destino.

Por exemplo, o cálculo pode ser representado por:

$$H = |\text{linha atual} - \text{linha destino}| + |\text{coluna atual} - \text{coluna destino}|$$

Dessa forma, o algoritmo consegue ter uma estimativa de quantos movimentos ainda serão necessários para chegar ao destino.

Movimentação do agente

O agente foi criado na classe Agent.

No momento em que é iniciado, ele recebe uma posição inicial e uma posição de destino. Essas informações são enviadas para o AStarPathfinder, que calcula o caminho.

Depois disso, o agente segue os pontos encontrados pelo algoritmo, passando de um tile para o próximo.

Os movimentos possíveis são:

- Cima;
- Baixo;
- Esquerda;
- Direita.

O agente não recebe comandos do jogador para realizar essa movimentação. Depois que o caminho é calculado, ele realiza o percurso automaticamente.

Obstáculos

Durante o cálculo do caminho, o algoritmo verifica se cada tile pode ser atravessado.

As paredes são consideradas como posições bloqueadas. Quando o algoritmo encontra uma parede, ela não é adicionada ao caminho.

Também é feita uma verificação para evitar que o agente tente acessar uma posição que esteja fora do mapa.

Com isso, o agente consegue encontrar outro caminho quando existe algum obstáculo entre ele e o destino.

Visualização do caminho

Para facilitar o teste da atividade, o caminho encontrado pelo A* foi mostrado diretamente no mapa.

Os tiles que fazem parte do caminho ficam destacados visualmente, permitindo acompanhar a rota calculada pelo algoritmo.

O destino também possui uma marcação diferente no cenário.



Teste realizado

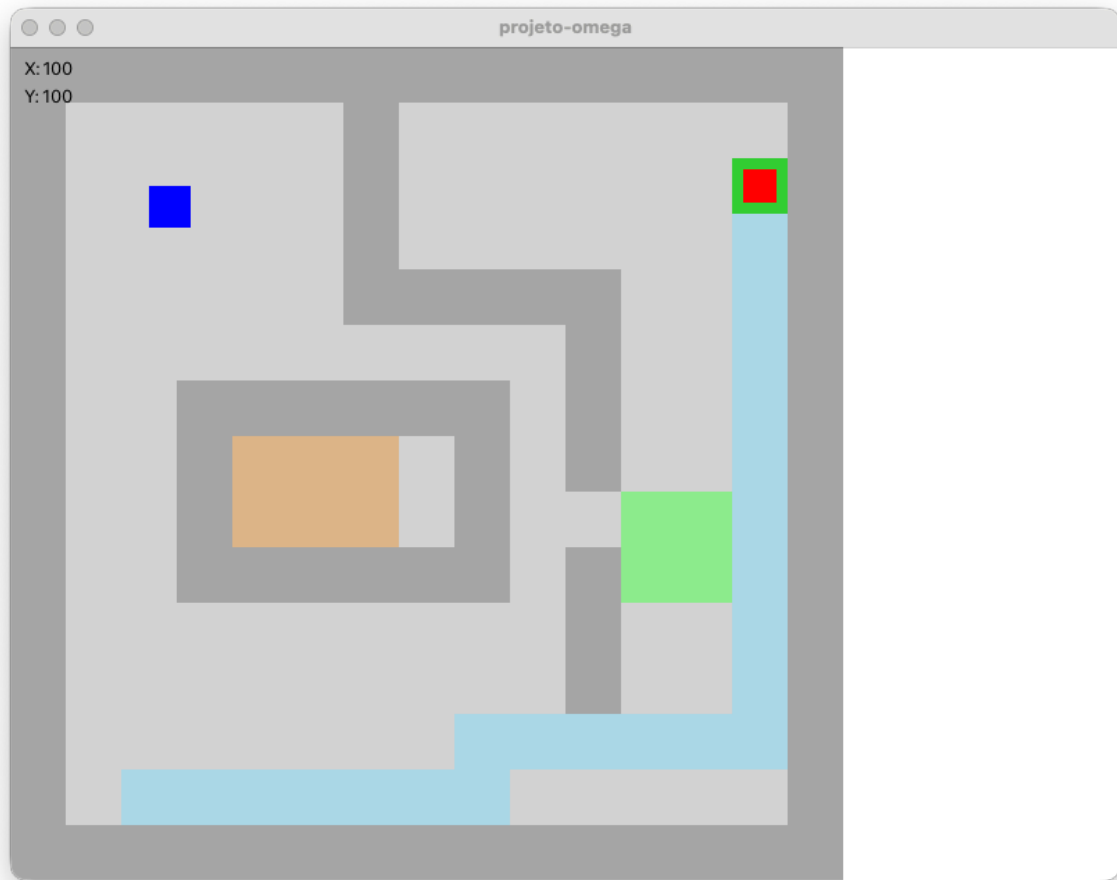
Depois da implementação, o projeto foi executado para verificar se o agente conseguia realizar o percurso corretamente.

No teste, o agente começou em uma posição definida no mapa e recebeu um destino fixo. Ao iniciar o jogo, ele calculou o caminho e começou a se movimentar automaticamente.

Foi possível verificar que o agente:

- encontrou um caminho até o destino;
- não atravessou as paredes;
- permaneceu dentro dos limites do mapa;
- seguiu os pontos do caminho na ordem correta;
- chegou ao destino;

- parou depois de alcançar o objetivo.



Classes utilizadas

Para realizar a atividade, foram utilizadas algumas das classes que já faziam parte do projeto e outras que foram adicionadas nessa etapa.

Tile

Representa cada tile do mapa e armazena informações como seu tipo, se pode ser atravessado e seu custo de movimentação.

TileMap

É responsável por organizar os tiles em uma matriz e representar o cenário do jogo.

AStarPathfinder

Contém a implementação do algoritmo A*. É responsável por analisar o mapa, calcular a heurística e encontrar o caminho até o destino.

Agent

Representa o personagem controlado pela Inteligência Artificial. Ele recebe o caminho calculado pelo A* e realiza a movimentação automaticamente.

GameLoop

É responsável por atualizar e desenhar os elementos do jogo durante a execução, incluindo o agente e o caminho encontrado.

Resultado

Depois dos testes, a implementação apresentou o resultado esperado. O agente conseguiu encontrar uma rota pelo mapa e seguir o caminho calculado até chegar ao destino.

A atividade também ajudou a integrar o sistema de navegação com o mapa que já havia sido desenvolvido nas etapas anteriores.

Essa implementação será importante para as próximas partes do Projeto Ômega, principalmente para a criação dos inimigos e de seus comportamentos dentro do laboratório.

Conclusão

Com esta atividade, foi possível colocar em prática o funcionamento do algoritmo A* dentro do jogo.

A principal mudança foi fazer com que o agente deixasse de depender de uma movimentação definida manualmente e passasse a calcular seu próprio caminho levando em consideração os obstáculos do cenário.

O sistema ainda pode ser melhorado nas próximas etapas, principalmente quando forem adicionados os inimigos, seus comportamentos e os sistemas de percepção por visão e som previstos para o projeto.